

Computer Architecture Project

Project Report:

“Traffic Light Controller Using Finite State Machine”

Traffic Light Controller Using Finite State Machine (FSM)

1. Introduction

Traffic control at busy intersections is a critical challenge in modern transportation systems. Poor traffic management can lead to congestion, delays, and accidents. To overcome these issues, automated traffic light systems based on digital logic are widely used.

This project focuses on the design and implementation of a Traffic Light Controller using a Finite State Machine (FSM). The controller manages traffic flow for four different roads by sequencing traffic lights using precise timing and safe transition rules. The system is implemented in Verilog HDL and verified using ModelSim, demonstrating key computer architecture and digital system design concepts.

2. Objectives of the Project

The main objectives of this project are:

- To design a Moore-type Finite State Machine
- To control four traffic signals without conflict
- To implement timing control using clock cycles
- To verify the design using ModelSim simulation
- To apply computer architecture theory to a real-world problem

3. System Specifications

3.1 Inputs

- **clk** – System clock (100 MHz, 10 ns period)
- **rst** – Active-high reset signal

3.2 Outputs

- **light_M1** – Main Road 1 traffic light
- **light_M2** – Main Road 2 traffic light
- **light_MT** – Main Through Road traffic light
- **light_S** – Side Road traffic light

Each output is a **3-bit encoded signal**.

4. Traffic Light Encoding

Binary Code	Light Color
3'b001	Green
3'b010	Yellow
3'b100	Red

This encoding ensures clarity and prevents ambiguous signal interpretation.

5. Clock and Timing Parameters

The controller operates at a 100 MHz clock frequency, where:

➤ **1 clock cycle = 10 ns**

Timing Parameters Used

parameter sec7 = 7; // 70 ns

parameter sec5 = 5; // 50 ns

parameter sec3 = 3; // 30 ns

parameter sec2 = 2; // 20 ns

6. Hardware Implementation

The Traffic Light Controller is implemented using synchronous digital hardware, designed in Verilog HDL. The hardware architecture follows a structured FSM-based approach.

Hardware Components Description

➤ **Clock Generator**

Provides a stable 100 MHz clock signal to synchronize all operations.

➤ **Reset Unit**

Initializes the system and forces the controller into a known safe state (State S1).

➤ **State Register (FSM Core)**

Stores the present state of the traffic controller and updates on each clock edge.

➤ **Counter Unit**

Controls the duration of each state by counting clock cycles according to timing parameters.

➤ **Next-State Logic**

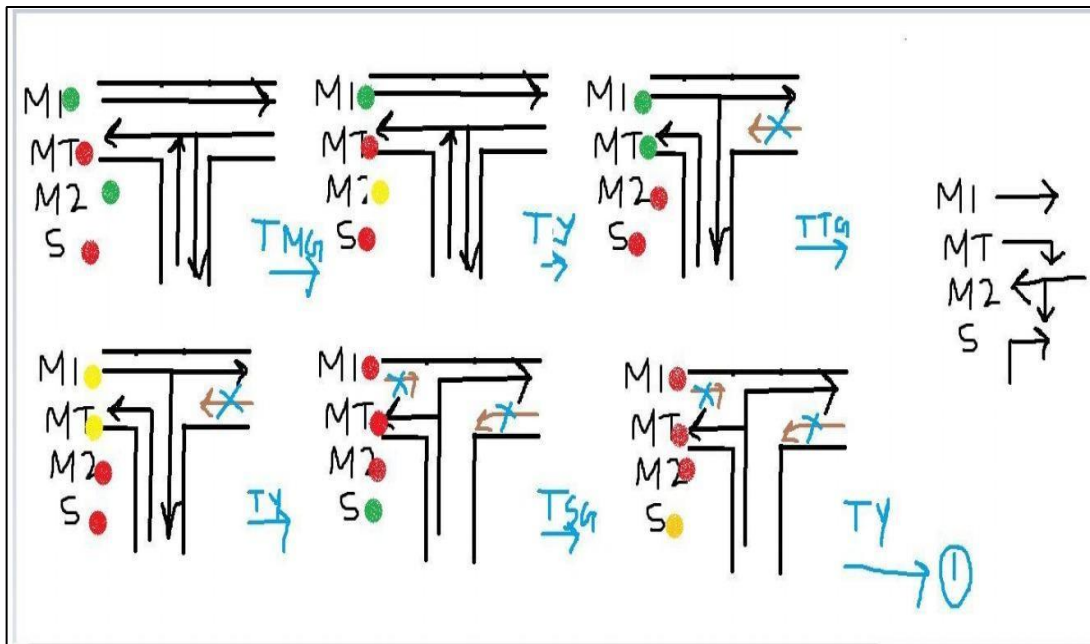
Determines the transition from one state to another based on the counter value.

➤ **Output Logic**

Generates traffic light signals based solely on the current state (Moore FSM).

7. Finite State Machine (FSM) Design

The controller uses a Moore Finite State Machine, where outputs depend only on the present state. This approach ensures stable outputs and glitch-free operation.



FSM States Overview

State	Description
S1	M1 and M2 Green
S2	M1 Green, M2 Yellow
S3	M1 and MT Green
S4	M1 and MT Yellow
S5	Side Road Green
S6	Side Road Yellow

After S6, the FSM returns to S1, completing one full traffic cycle

8. Timing Analysis

State	Clock Cycles	Time (ns)	Percentage
S1	7	70	33.3%
S2	2	20	9.5%
S3	5	50	23.8%
S4	2	20	9.5%
S5	3	30	14.3%
S6	2	20	9.5%
Total	21	210 ns	100%

This timing ensures smooth vehicle flow and safe light transitions.

9. Verilog HDL Implementation

Main Traffic Light Controller Module

The main module implements the FSM, counter, and output logic using Verilog HDL.

// Traffic_Light_Controller.v

Code:

```
`timescale 1ns / 1ps

module Traffic_Light_Controller(

    input clk,

    input rst,

    output reg [2:0] light_M1,

    output reg [2:0] light_S,

    output reg [2:0] light_MT,

    output reg [2:0] light_M2

);

// State definitions

parameter S1 = 0, S2 = 1, S3 = 2, S4 = 3, S5 = 4, S6 = 5;

// Timing parameters (in clock cycles)

parameter sec7 = 7, sec5 = 5, sec2 = 2, sec3 = 3;

reg [3:0] count;

reg [2:0] ps; // Present state
```

```
// State transition logic

always @(posedge clk or posedge rst)

begin

    if (rst == 1'b1)

        begin

            ps <= S1;

            count <= 4'b0;

        end

    else

        begin

            case (ps)

                S1:

                    if (count < sec7)

                        begin

                            ps <= S1;

                            count <= count + 1;

                        end

                    else

                        begin

                            ps <= S2;

                            count <= 4'b0;

                        end

                S2:
```

```
    if (count < sec2)

    begin

        ps <= S2;

        count <= count + 1;

    end

    else

    begin

        ps <= S3;

        count <= 4'b0;

    end

    S3:

    if (count < sec5)

    begin

        ps <= S3;

        count <= count + 1;

    end

    else

    begin

        ps <= S4;

        count <= 4'b0;

    end

    S4:

    if (count < sec2)
```

```
begin

    ps <= S4;

    count <= count + 1;

end

else

begin

    ps <= S5;

    count <= 4'b0;

end

    S5:

if (count < sec3)

begin

    ps <= S5;

    count <= count + 1;

end

else

begin

    ps <= S6;

    count <= 4'b0;

end

    S6:

if (count < sec2)

begin
```

```

        ps <= S6;

        count <= count + 1;

    end

    else

    begin

        ps <= S1;

        count <= 4'b0;

    end

    default: ps <= S1;

endcase

end

end

// Output logic

always @(ps)

begin

    case (ps)

        S1: // State 1

        begin

            light_M1 <= 3'b001; // Green on M1

            light_M2 <= 3'b001; // Green on M2

            light_MT <= 3'b100; // Red on MT

            light_S <= 3'b100; // Red on S

        end

    end

end

```

S2: // State 2

begin

light_M1 <= 3'b001; // Green on M1

light_M2 <= 3'b010; // Yellow on M2

light_MT <= 3'b100; // Red on MT

light_S <= 3'b100; // Red on S

end

S3: // State 3

begin

light_M1 <= 3'b001; // Green on M1

light_M2 <= 3'b100; // Red on M2

light_MT <= 3'b001; // Green on MT

light_S <= 3'b100; // Red on S

end

S4: // State 4

begin

light_M1 <= 3'b010; // Yellow on M1

light_M2 <= 3'b100; // Red on M2

light_MT <= 3'b010; // Yellow on MT

light_S <= 3'b100; // Red on S

end

S5: // State 5

```
begin

    light_M1 <= 3'b100; // Red on M1

    light_M2 <= 3'b100; // Red on M2

    light_MT <= 3'b100; // Red on MT

    light_S <= 3'b001; // Green on S

end

S6: // State 6

begin

    light_M1 <= 3'b100; // Red on M1

    light_M2 <= 3'b100; // Red on M2

    light_MT <= 3'b100; // Red on MT

    light_S <= 3'b010; // Yellow on S

end

default: // Default (should not occur)

begin

    light_M1 <= 3'b000;

    light_M2 <= 3'b000;

    light_MT <= 3'b000;

    light_S <= 3'b000;

end

endcase

end

endmodule
```

Testbench Module

The testbench verifies correct functionality by generating clock, reset, and observing outputs.

// **Traffic_Light_Controller_TB.v**

Code:

```
`timescale 1ns / 1ps

module Traffic_Light_Controller_TB;

    // Inputs
    reg clk;
    reg rst;

    // Outputs
    wire [2:0] light_M1;
    wire [2:0] light_S;
    wire [2:0] light_MT;
    wire [2:0] light_M2;

    // Instantiate the Unit Under Test (UUT)
    Traffic_Light_Controller dut (
        .clk(clk),
        .rst(rst),
        .light_M1(light_M1),
        .light_S(light_S),
        .light_MT(light_MT),
```

```

    .light_M2(light_M2)

);

    // Clock generation

initial

begin

    clk = 1'b0;

    // Note: The original testbench uses 100,000,000ns period (0.1s)

    // This is VERY SLOW for simulation. I'm using 10ns period for faster simulation

    forever #5 clk = ~clk; // 10ns period = 100MHz clock

end

    // Stimulus process

initial

begin

    // Initialize inputs

    rst = 1'b0;

    // Apply reset

    #10;

    rst = 1'b1;

    #20;

    rst = 1'b0;

    // Run simulation for enough cycles to see all states

    // Each state takes: S1(7), S2(2), S3(5), S4(2), S5(3), S6(2) = 21 cycles total

    // With 10ns clock period, 21 cycles = 210ns

    // Let's run for 500ns to see multiple cycles

```

```
#500;

    // Apply reset again to see reset functionality

    rst = 1'b1;

    #20;

    rst = 1'b0;

    // Run for another cycle

    #300;

    $finish;

end

// Monitor to display results in console

initial

begin

    $monitor("Time=%0t: State Outputs - M1=%b, M2=%b, MT=%b, S=%b",

        $time, light_M1, light_M2, light_MT, light_S);

    // Create waveform file

    $dumpfile("traffic_wave.vcd");

    $dumpvars(0, Traffic_Light_Controller_TB);

end

endmodule
```

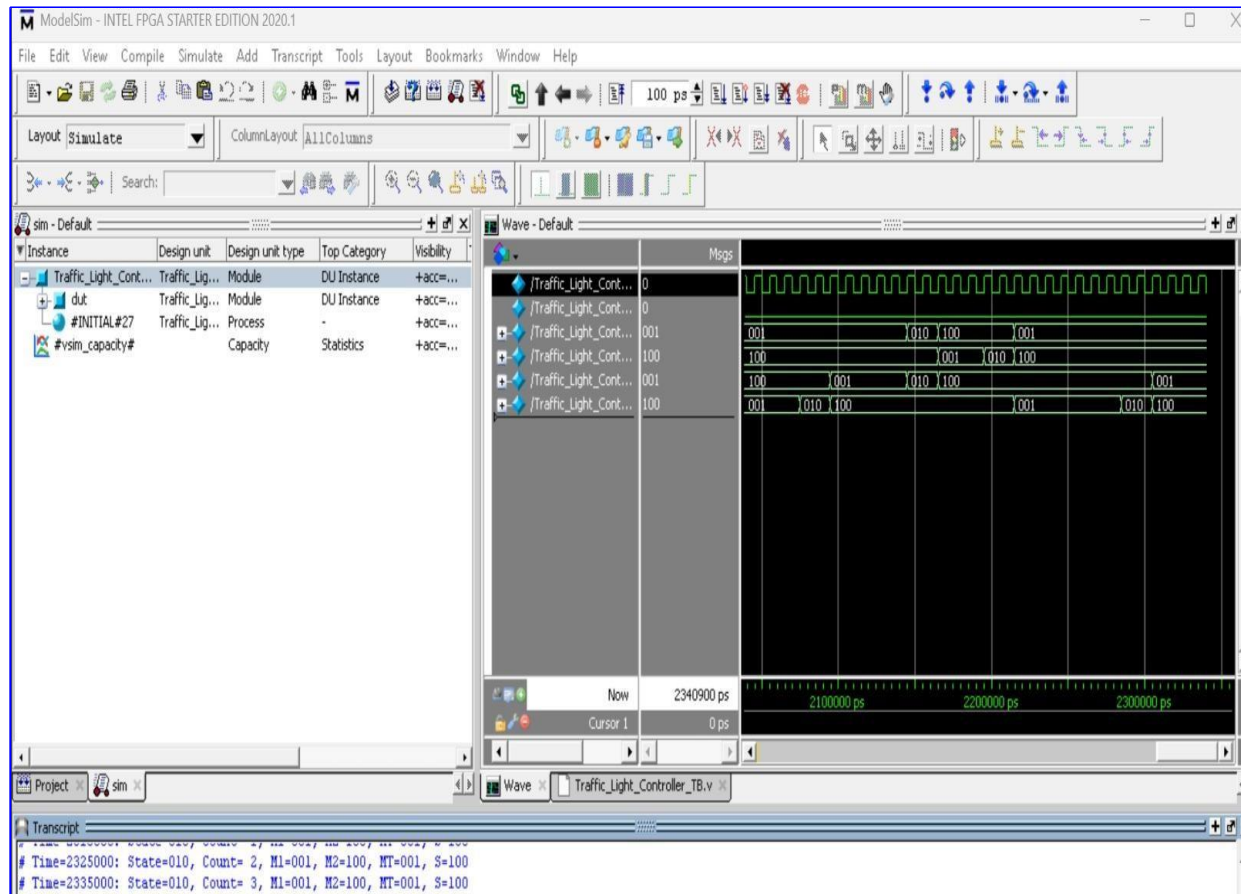
10. Simulation and Output Verification

The design was simulated using ModelSim (Intel FPGA Starter Edition).

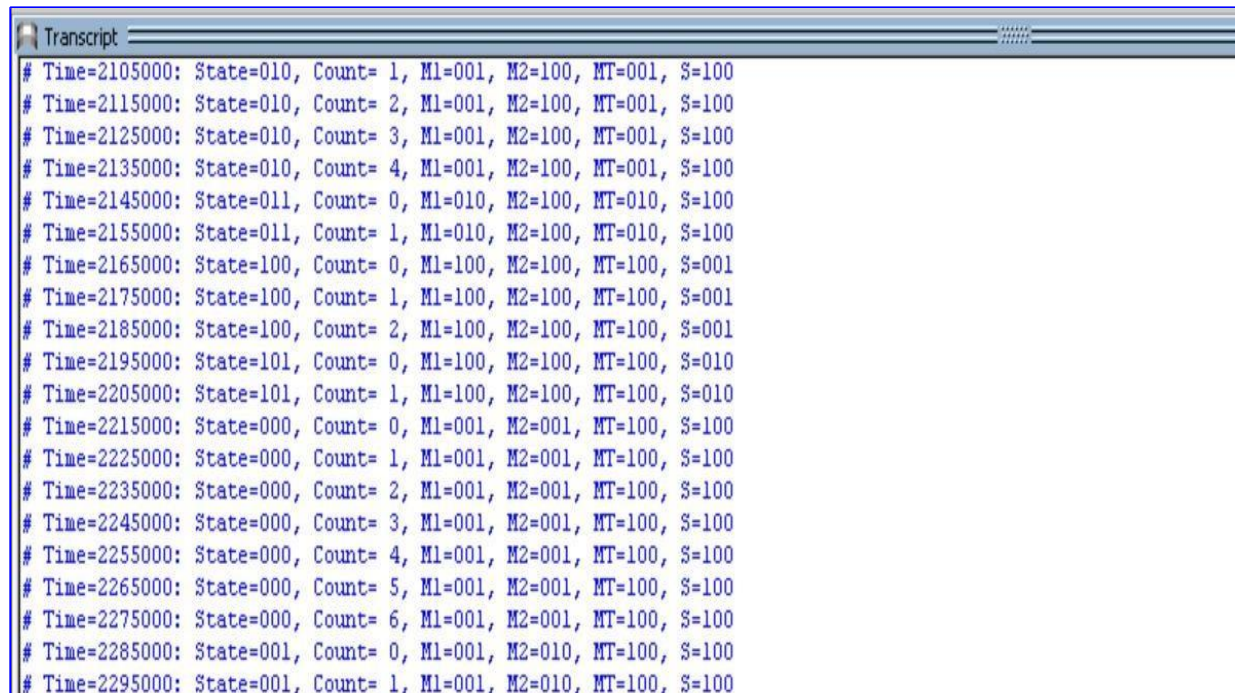
Simulation Observations

- Correct FSM state transitions
- Accurate timing based on clock cycles
- No conflicting green signals
- Reset returns system to safe initial state

ModelSim Waveform Output Screenshot



12. Console Output (Transcript Results)



```
# Time=2105000: State=010, Count= 1, M1=001, M2=100, MT=001, S=100
# Time=2115000: State=010, Count= 2, M1=001, M2=100, MT=001, S=100
# Time=2125000: State=010, Count= 3, M1=001, M2=100, MT=001, S=100
# Time=2135000: State=010, Count= 4, M1=001, M2=100, MT=001, S=100
# Time=2145000: State=011, Count= 0, M1=010, M2=100, MT=010, S=100
# Time=2155000: State=011, Count= 1, M1=010, M2=100, MT=010, S=100
# Time=2165000: State=100, Count= 0, M1=100, M2=100, MT=100, S=001
# Time=2175000: State=100, Count= 1, M1=100, M2=100, MT=100, S=001
# Time=2185000: State=100, Count= 2, M1=100, M2=100, MT=100, S=001
# Time=2195000: State=101, Count= 0, M1=100, M2=100, MT=100, S=010
# Time=2205000: State=101, Count= 1, M1=100, M2=100, MT=100, S=010
# Time=2215000: State=000, Count= 0, M1=001, M2=001, MT=100, S=100
# Time=2225000: State=000, Count= 1, M1=001, M2=001, MT=100, S=100
# Time=2235000: State=000, Count= 2, M1=001, M2=001, MT=100, S=100
# Time=2245000: State=000, Count= 3, M1=001, M2=001, MT=100, S=100
# Time=2255000: State=000, Count= 4, M1=001, M2=001, MT=100, S=100
# Time=2265000: State=000, Count= 5, M1=001, M2=001, MT=100, S=100
# Time=2275000: State=000, Count= 6, M1=001, M2=001, MT=100, S=100
# Time=2285000: State=001, Count= 0, M1=001, M2=010, MT=100, S=100
# Time=2295000: State=001, Count= 1, M1=001, M2=010, MT=100, S=100
```

This verifies accurate FSM execution and output generation.

12. Real-World Applications

- Four-way traffic intersections
- T-junction traffic systems
- Industrial and factory vehicle control
- Smart city traffic automation

13. Conclusion

The Traffic Light Controller project successfully demonstrates the use of a six-state Moore Finite State Machine to manage traffic signals efficiently. The Verilog-based design, combined with ModelSim simulation, validates correct timing, safe transitions, and reliable operation.

This project reflects a strong understanding of computer architecture principles, including FSM design, sequential logic, and hardware timing control. The system provides a solid foundation for future enhancements such as sensor-based control and intelligent traffic systems.

END!